第18章 动态规划DP

18.2 背包型DP

18.2.1 背包型DP的定义

背包型DP用来解决这样的问题：给定一个有体积的背包和一些物品（每个物品有体积和价值），以及限制条件（所选物品体积不能超过总背包体积），求解最优

物品选择（物品总价值最大）方案。

注意：上面带括号部分只是为了便于读者理解举了其中一个例子，题目千变万化，物品属性和限定条件要视具体情况而定。

18.2.2 背包的多种类型

01 背包（每种物品只能用一次）、完全背包（每种物品可以用无限次）、多重背包（每种物品可以用若干次）、混合背包、分组背包。通过一些例题来描述这些类型以及解决方法。

例题18-14：采药（NOIP 2005 普及组）

【题目描述】

n 颗草药，采每颗草药所需时间为 $t[i]$,也有一定价值 $v[i]$ ，求不超过总时间 T 所能采得的最大价值。

【输入】

T 和 n

n 行，每行两个整数 $t[i]$ 和 $v[i]$

【样例输入】

```
70 3
71 100
69 1
1 2
```

【样例输出】

```
3
```

【数据范围】

$1 \leq T \leq 1000$

$1 \leq n \leq 100$

$1 \leq t[i], v[i] \leq 100$

【题目分析】

这是比较基础的背包问题，联系上面给出的背包型 DP 的定义，我们可以把草药看成物品，所需采摘时间看成物品体积，总时间看成总体积，那么问题就变成在 n

个物品中任取，不超过 T 体积所能取得的最大价值。

这类问题用动态规划（以下简称DP) 如何来解决？

首先背包类问题的问题阶段非常明显，就是对每一个物品进行选择，取或者不取，并记录这样的选取所对应的最优值。

这个最优值我们给它一个概念：**状态** ，采药问题中，状态就是 $f[i][j]$ ，它表示在前 i 颗草药中任取，正好花费 j 时间能够采得的最大价值，

如何求得每一个状态值，换言之如何进行 **状态转移** ，它应该发生在选取上一个物品和选取这一个物品之间，也就是说如何利用已经求得的 $f[i-1][?]$ 求取还未求

得的 $f[i][?]$ ， 考虑第 i 颗草药的选取（采摘第 i 颗草药时间为 $t[i]$ ，价值为 $v[i]$ ），只有两种情况选或不选：如果选，并且选完形成状态 $f[i][j]$ 的话，那么选之前

的状态就应该是 $f[i-1][j-t[i]]$，并且选完之后的价值会增加 $v[i]$ ；如果不选 ，那么之前存在的状态 $f[i-1][j]$ ，可以直接转化为 $f[i][j]$ ，综上所述：

$$ f[i][j]=\left{ \begin{array}{rcl} f[i-1][j] & {不采摘第i颗草药}\ f[i-1][j-t[i]]+v[i] & {采摘第i颗草药} \end{array} \right. $$

然后考虑**初值**，既然求最大值，默认初值就是最小值, 前0颗草药只能构成0的时间，这时候的价值当然也是0，$f[0][0]=0$，**终值**就应该是所有的 $f[n][?]$ 找个最大

值。

用**表格填充**的方式再来理解下：

如下图表格的行（前1颗、前2颗.....）表示前几颗草药放完后的状态值，列表示背包容量，这道题目里就是花费的时间值，单元格内容表示相应的状态值 $f[i][j]$ 。

虚线框中 7（$f[2][4]$）, 表示前 2 颗草药中任取，花费 4 的时间能够取得的最大值，这个7的来源应该是上一行虚线框中的2（$f[1][1]$）, 表示前1颗任取，花费 1 的

时间能够取得价值 2 ，那么第 2 颗草药如果选进来，花费 $1+3$ 的时间自然能取到 $2+5$ 的价值。

实线框中 11（$f[5[5]$）和 16（$f[5][8]$）反应了另外一种情况，按照上一段的描述，$f[5][5]$ 应该由 $f[4][0]$ +9 得来，$f[4][0]$ 在图中是 0，因此 $f[5][5]$ 是 9，但为什

么还是填 11 呢，因为这是选了第 4 颗草药的最优值，但注意我们的状态描述，既然是任取，当然可以不选，$f[5][5]$ 直接拿到$f[4][5]$ 的值 11 ，比 9 来得大，

$f[5][8]$ 也遇到类似的情况。

			背包容量（本题是时间）													
			0	1	2	3	4	5	6	7	8	9	10	11	12	13
t[i]	v[i]	前0颗	0	Min	Min	Min	Min	Min	Min	Min	Min	Min	Min	Min	Min	Min
1	2	前1颗	0	2	Min	Min	Min	Min	Min	Min	Min	Min	Min	Min	Min	Min
3	5	前2颗	0	2	Min	5	7	Min	Min	Min	Min	Min	Min	Min	Min	Min
4	9	前3颗	0	2	Min	5	9	11	Min	14	16	Min	Min	Min	Min	Min
5	9	前4颗	0	2	Min	5	9	11	9	14	16	Min	20	Min	23	25

虚线框、实线框中的操作概括起来就是在两种可能的状态来源中取个最大值，公式描述如下：
$f[i][j] = max(f[i-1][j-t[i]] + v[i], f[i-1][j])$

填完整个表格也对应了运行时间，因此**时间复杂度**就是$O(n \times T)$($草药数量 \times 时间容量 \times 决策数量$，时间复杂度算式中出现常数的乘积可直接忽略，此题决策数量为

2），这里的**决策**表示状态转移来源的数量。

【核心代码】

```
for (int i=1;i<=n;i++)
        for (int j=1;j<=sum;j++){
            f[i][j]=f[i-1][j];//第i颗草药不取，直接继承上一行的状态值
            if (j>=a[i]) //第i颗草药取，时间容量要足够采第i颗草药、防止数组下标越界
                f[i][j]=max(f[i][j],f[i-1][j-a[i]]+b[i]);
        }
```

【再分析】

从上面的公式可以看出，下一行的状态默认可以继承上一行的，既然要“抄”下来的，何不并成一行呢，把原有的二维状态 $f[i][j]$ 压缩成一维状态 $f[j]$ ，状态转移的

其中一个操作（当前这颗草药不放入） $f[i][j] = f[i-1][j]$ 自然就实现了，那么另一个操作（放入当前这颗草药）$f[i][j] = f[i-1][j-t[i]] + v[i]$ 怎么实现，是

按照列升序求取还是降序求取，还是两者兼可，我们继续从填表中找寻答案，从下图（由前0颗最优值得到前1颗最优值）可以看出只能降序求取状态值才行，升序

会让本次刚求得的状态又去产生新的状态值，相当于草药被多次取用。

			背包容量（本题是时间）													
			0	1	2	3	4	5	6	7	8	9	10	11	12	13
t[i]	v[i]	前0颗	0	2	4	6	8	10	12	14	16	18	20	22	24	26
1	2	前1颗														

			背包容量（本题是时间）													
			0	1	2	3	4	5	6	7	8	9	10	11	12	13
t[i]	v[i]	前0颗	0	2	Min	Min	Min	Min	Min	Min	Min	Min	Min	Min	Min	Min
1	2	前1颗														

状态转移公式：

$f[j] = max(f[j], f[j-t[i]] + v[i])$

初值、终值、时间复杂度和之前二维状态的一致。

【核心代码】

```
for (int i=1;i<=n;i++)
        for (int j=T;j>=t[i];j--) //用一维的状态后，只能降序遍历，避免新状态值去影响别的状态
            f[j]=max(f[j],f[j-t[i]]+v[i]);
```

【完整代码_二维状态】

```
#include<bits/stdc++.h>
using namespace std;
const int inf=-1e9;
int a[200],b[200],f[200][20000];
int main(){
    int T,n;
    int sum=0;
    scanf("%d%d",&T,&n);
    for (int i=1;i<=n;i++){
        scanf("%d%d",&a[i],&b[i]);
        sum+=a[i];
    }
    for (int j=1;j<=sum;j++)
            f[0][j]=inf;
    f[0][0]=0;
    for (int i=1;i<=n;i++)
        for (int j=1;j<=sum;j++){
            f[i][j]=f[i-1][j];
            if (j>=a[i])
            f[i][j]=max(f[i][j],f[i-1][j-a[i]]+b[i]);
        }
    int ans=0;
    for (int j=1;j<=T;j++){
        ans=max(ans,f[n][j]);
    }
    printf("%d\n",ans);
}
```

【完整代码_一维状态】

```
#include<bits/stdc++.h>
using namespace std;
const int Min=-1e9;
int t[110],v[110],f[1100];
int main(){
    int T,n;
    scanf("%d%d",&T,&n);
    for (int i=1;i<=n;i++)
        scanf("%d%d",&t[i],&v[i]);
    for (int j=1;j<=T;j++)
        f[j]=Min;
    f[0]=0;
    for (int i=1;i<=n;i++)
```

```
        for (int j=T;j>=t[i];j--)
            f[j]=max(f[j],f[j-t[i]]+v[i]);
    int ans=-1;
    for (int j=0;j<=T;j++)
        ans=max(ans,f[j]);
    printf("%d\n",ans);
}
```

例题18-15：硬币问题（zhima 780）

【题目描述】

n 种面值的硬币，每种的面值为 $a[i]$，每种无限，求形成 S 面值的需要的最少硬币数量和最多硬币数量。

【题目分析】

和上一个例题相比，这里的硬币相当于草药，面值相当于时间，但有个不同点：每种硬币可以无限次使用，不像草药只能使用一次。回想上一个图的升序遍历错误演示，不正是解决物品可以无限次使用的方法吗。另外本题有两个问题，构成S面值所需的最少数量和最多数量，但本质是相同的。于是一维的状态以及转移方程就清晰了。

状态：$f[j]$ 表示形成 j 面值所需的最少硬币数量，$g[j]$ 表示形成 j 面值所需的最多硬币数量。

状态转移：

$f[j] = min(f[j], f[j-a[i]]+1)$

$g[j] = max(g[j], g[j-a[i]]+1)$

初值：求最大值，默认初值就应该尽可能小，如果不够小通过一些事实上不可行的转移 $+1$ 后就会比正解更优，从而产生错误。求最小值，默认初值就应该尽可能大，同上。形成 0 面值当然只能用 0 个硬币，因此 $f[0]=g[0]=0$

终值：很显然就是 $f[S]$，$g[S]$

时间复杂度：$O(n \times S)$

【完整代码】

```
#include<bits/stdc++.h>
using namespace std;
const int inf=20000;
int a[200],f[20000],g[20000];
int main(){
    int n,s;
    scanf("%d",&n);
    scanf("%d",&s);
    for (int i=1;i<=n;i++)
        scanf("%d",&a[i]);
    for (int i=1;i<=s;i++){
        f[i]=inf;
        g[i]=-inf;
    }

    f[0]=g[0]=0;
```

```
    for (int i=1;i<=n;i++)
        for (int j=a[i];j<=s;j++){
            f[j]=min(f[j],f[j-a[i]]+1);
            g[j]=max(g[j],g[j-a[i]]+1);
        }
    printf("%d\n%d\n",f[s],g[s]);
}
```

例题18-16：提款机 (POJ1276)

【题目描述】

n 种纸币，每种 $a[i]$ 个，面值是 $b[i]$，求能够形成的最接近 S 面值的面值，有多组数据

【数据范围】

$n \le 10$

$a[i] \le 1000$

$b[i] \le 100000$

【样例输入】

```
735 3  4 125  6 5  3 350
633 4  500 30  6 100  1 5  0 1
735 0
0 3  10 100  10 50  10 10
```

【样例输出】

```
735
630
0
0
```

【题目分析】

和前两道例题相比：区别一，每个物品既不是取用一次，也不是无限次使用，而是有限的若干次；区别二，状态数组值的类型用 $bool$ 即可 ，表示能否形成某一个面值。

第一种方法就是把每种物品的若干个，看成独立的若干个物品，然后用例题1中背包思路做。但关注到数据范围，时间复杂度=物品数量 $\times$ 面值容量 ($n \times a[i] \times b[i]$)，会达到 $10 \times 10^3 \times 10^5$ 显然会超时。

第二种方法利用二进制拆分使物品数量减少，然后再用例题1的背包思路做。如何进行二进制拆分，举个例子：如果某一种物品有 15 (二进制 1111) 个，我们看成四个捆绑起来的新物品，分别是原有物品 8 (二进制 1000) 个、4 (二进制 100) 个、2 (二进制 10) 个、1 (二进制 1) 个，利用这四个新物品任意组合就能形成 0 个~ 15 个，因为 0 到 15 转成二进制无非就是 1000 、100 、10 、1 四个二进制位用或不用的组合。

再看一个例子：如果某一种物品数量为 17 个，又该怎么操作，比 16（二进制 10000）大，又比 32（二进制 100000）小，只要把超出最大二进制权值的部分看成一个新物品，所以 17 可以看成 8、4、2、1、2 这五个新物品。

有了上述二进制拆分转化的技巧，我们可以设计DP思路。

状态：$f[j]$ 表示前 i 种纸币里任意取，能否形成 j 面值。

状态转移：$f[j] = f[j] \mid f[j-b[i]]$。

初值：不用任何纸币只能形成 0 的面值，因此 $f[0] = 1$。

终值：找能形成的接近 S 的最大面值。

【完整代码】

```
#include<cstdio>
#include<cstring>
using namespace std;
int c,n,f[101000],b[200],cas;
struct node{
    int num,v;
};
node a[110];
void erfen(int v,int total){
    int i=1;
    while (total>=i){
        total-=i;
        b[++c]=i*v;
        i=i*2;
    }
    if (total>0) b[++c]=total*v;
}
int main(){
    while (~scanf("%d%d",&cas,&n)){
        memset(f,0,sizeof(f));
        c=0;
        int j;
        for (int i=1;i<=n;i++){
            scanf("%d%d",&a[i].num,&a[i].v);
            erfen(a[i].v,a[i].num);
        }
        f[0]=1;
        for (int i=1;i<=c;i++)
            for (j=cas;j>=b[i];j--){
                if (f[j-b[i]]) f[j]=1;
            }
        for (j=cas;j>=0;j--)
            if (f[j]) break;
        printf("%d\n",j);
    }
}
```

例题18-17 塔（洛谷1651）

【题目描述】

小明很喜欢摆积木，现在他正在玩的积木是由 N 个木块组成的，他想用这些木块搭出两座高度相同的塔，一座塔的高度是搭建它的所有木块的高度和，并且一座塔至少要用一个木块。每个木块只能用一次，也可以不用。目前已知每块木块的高度，小明想知道在最终两个塔的高度相同的情况下，他所能搭的塔的最大高度是多少，你能帮助他吗？

【输入】

第一行为一个整数 N，表示木块个数。

第二行是 N 个整数，表示 N 块木块的高度。

【数据规模】

对于 100% 的数据，$N \leq 50$ ，每块木块的高度 h 满足 $1 \leq h \leq 500000$，所有木块的高度总和 $H \leq 500000$ 。

【输出】

仅一个整数，表示能搭建的塔的最大高度，若不能搭建两座相同高度的塔，则输出“-1”。

【样例输入】

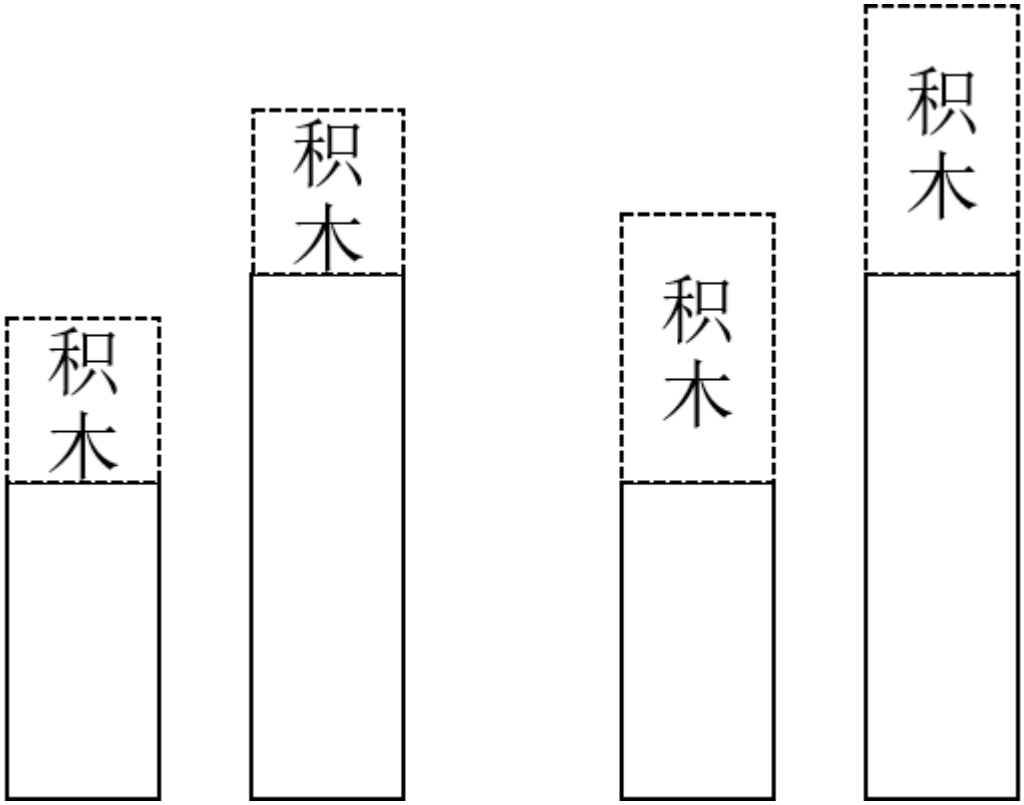
```
3
2 3 5
```

【样例输出】

```
5
```

【题目分析】

背包物品是积木，背包容量又是什么？我们可以画个图帮助理解。



从图上可以看出我们只要知道两座塔分别的高度，就能描述状态的改变，状态数组就是 $f[i][j][k]$ 表示前i个积木任意取用，能否形成双塔高度分别为 j 和 k 的状态，根据下标计算数组内存至少就需要 $(H \div 2 \times H \div 2 \times 1 \div 8)$ M，显然会超过内存限制。

再次观察图形，在设计状态的时候我们只要确保构造的状态能够满足状态转移所需要的信息，状态转移中只要关注新积木的放入，对此过程有影响的主要是两个塔的高度差，有了高度差，只要再能描述其中一个塔的高度，两个塔的高度信息就都有了。

因此可以设计如下状态：$f[i][j]$ 表示前i个积木高度差是j的最高塔的高度值。

状态转移：

- 积木放入较高塔时

  $f[i][j+a[i]] = max(f[i][j+a[i]], f[i-1][j]+a[i])$
- 积木放入较低塔时

  当积木高度小于j时 $f[i][j-a[i]] = max(f[i][j-a[i]], f[i-1][j])$

  当积木高度大于等于j时 $f[i][a[i]-j] = max(f[i][a[i]-j], f[i-1][j]+a[i]-j)$

初值：状态数组 f 初始成最大值，$f[0][0]=0$

终值：如果 $f[n][0]$ 没有获得过值说明无解，否则输出 $f[n][0]$

时间复杂度：$O(n \times H)$

另外这道题目可以使用**滚动数组**，它的适用题型要符合第i层（阶段）状态只和第 $i-1$ 层（阶段）状态有关，实现依靠不断重复这两句 $\{f[c][?]=f[1-c][?],$

$c=1-c\}$，详情见下方完整代码。

【完整代码】

```
#include<bits/stdc++.h>
using namespace std;
const int inf=-1e9;
int a[100],f[2][1000010];
int main(){
    int n;
    int sum=0;
    scanf("%d",&n);
    for (int i=1;i<=n;i++){
        scanf("%d",&a[i]);
        sum+=a[i];
    for (int i=0;i<=1;i++)
        for (int j=0;j<=sum;j++)
            f[i][j]=inf;
    f[0][0]=0;
    int c=0;
    for (int i=1;i<=n;i++){
        c=1-c;
        for (int j=0;j<=sum;j++){
            f[c][j]=max(f[c][j],f[1-c][j]);
            f[c][j+a[i]]=max(f[c][j+a[i]],f[1-c][j]+a[i]);
            if (j>=a[i])
                f[c][j-a[i]]=max(f[c][j-a[i]],f[1-c][j]);
            else
                f[c][a[i]-j]=max(f[c][a[i]-j],f[1-c][j]+a[i]-j);
        }
    }
    if (f[c][0]<=0) printf("-1\n");else printf("%d\n",f[c][0]);
}
```

例题18-18：三角牧场 (POJ1948)

$1S\ 30M$

【题目描述】

给定 N ($3 \leq N \leq 40$)根长度的木棍 Li ($1 \leq Li \leq 40$)，每根木棍都用完的情况下，求能围成的三角形中面积最大的那个输出：乘以 100 取整，如果无解输出 -1

。提示：海伦公式为 $S=\sqrt{P(P-a)(P-b)(P-c)}$ (P 是周长的一半)。

【样例输入】

```
5
1
1
3
3
4
```

【样例输出】

```
692
```

【题目分析】

这道题目的背包物品显然是小木棍，但背包容量没有那么明朗，小木棍要么不放、要么放入三边的任意一条，能否使用这样的状态描述 $dp[i][j][k][L]$ 表示前 i 个小

木棍任取，能否形成三条边分别为 j，k，L 的三角形，但存在一个问题放入小木棍没法预测后续全部放完后能否成功构成三角形，于是要换一种状态描述（状态

数组保持不变）：前 i 个小木棍任取，能否形成三条边分别为 j , k , L 。至于能否构成三角形，等所有 n 条边任取完后再判断就行了。再关注到状态数组内存要开

到 $(40 \times 800 \times 800 \times 800 \times 1 \div 8)$ M，会超过内存限制，因此状态数组要缩减，因为总长度一定，只要找出哪些木棍放在其中两条边，剩下的木棍自然放在第

三条边了，最终的**状态**及其描述：$dp[i][j][k]$表示前 i 个木棍形成边 j 、k 是否可能。

状态转移可以分类讨论（小木棍长度用 a 数组来存放）：

- 放入第三边状态来自 $dp[i-1][j][k]$
- 放入 j 状态来自 $dp[i][j-a[i]][k]$
- 放入 k 状态来自 $dp[i][j][k-a[i]]$

初值：$dp[0][0][0]=1$

思考：上述是放入0个木棍作为初值，如果把放入1个木棍作为初值，初值应该是怎么样的，是不是 $dp[1][a[1]][0]=1$ 和 $dp[1][0][a[1]]=1$ ，这样就忽略了放入第

三边的情况，还需要加入 $dp[1][0][0]=1$ 。在实际解题中，大家往往会把注意力集中在设计状态及状态转移，但千万不能忽视初值。

终值：在所有满足值为1的 $dp[n][?][?]$ 对应三角形面积中，找一个最大值。

时间复杂度（木棍总长为L）：$O(N \times (L \div 2)^2)$

【完整代码】

```
#include<cstdio>
#include<algorithm>
#include<cmath>
using namespace std;
int n,tot,a[41],ans;
bool dp[41][1000][1000];
bool cans(int j,int k){
    int t=tot-j-k;
    if (j+k>t&&j+t>k&&k+t>j) return 1;
    return 0;
}
int deal(int j,int k){
    double p=tot/2.0;
    return (int)100*sqrt(p*(p-j)*(p-k)*(p-(tot-j-k)));
}
```

```
int main(){
    scanf("%d",&n);
    for (int i=1;i<=n;i++){
        scanf("%d",&a[i]);
        tot+=a[i];
    }
    dp[0][0][0]=1;
    for (int i=1;i<=n;i++)
        for (int j=0;j<=tot/2;j++)
            for (int k=0;j+k<=tot;k++){
                dp[i][j][k]=dp[i-1][j][k];
                if (j>=a[i]) dp[i][j][k]=dp[i][j][k]||dp[i-1][j-a[i]][k];
                if (k>=a[i]) dp[i][j][k]=dp[i][j][k]||dp[i-1][j][k-a[i]];
            }
    ans=-1;
    for (int j=0;j<=tot/2;j++)
        for (int k=0;j+k<=tot;k++){
            if (cans(j,k))
                if (dp[n][j][k])ans=max(ans,deal(j,k));
        }
    printf("%d\n",ans);
}
```

例题18-19 挤牛奶（ZHIMA 779）

【题目描述】

n（偶数）头牛，每头牛的奶量为 $a[i]$ ，选一半的头数，使奶量最接近一半的总奶量。

【题目分析】

和传统背包相比，区别在于限定了选用的物品数，上一题就说到过，构造状态时一定要看能否描述状态转移时所需要的信息，于是我们可以在状态数组中加一个维

度描述已经选了几头。

状态：$dp[j][v]$ 表示前 i 头牛任选 j 头，能否取得 v 的奶量

状态转移：$dp[j][v]=dp[j-1][v-a[i]]$，这里计算状态的顺序很重要，j 变量的遍历只能是降序，否则新产生的状态值会去影响别的状态值。

初值：$dp[0][0]=1$

终值：在 $dp[n/2][X]==1$ 中找一个最接近一半奶量一半的 X ，再运算得出题目要求的答案

【完整代码】

```
#include<bits/stdc++.h>
using namespace std;
int a[110],dp[110][2100];
int main(){

    int n;
```

```
    scanf("%d",&n);
    int s=0;
    for (int i=1;i<=n;i++){
        scanf("%d",&a[i]);
        s+=a[i];
    }
    int stmp=s;
    s=s/2;
    int m=n/2;
    dp[0][0]=1;
    for (int i=1;i<=n;i++)
        for (int j=m;j>=1;j--)
            for (int v=a[i];v<=s;v++)
                if (dp[j-1][v-a[i]]) dp[j][v]=1;
    for (int v=s;v>=0;v--)
        if (dp[m][v]) {
            printf("%d\n",abs(stmp-v-v));
            break;
        }
}
```

例题18-20：太空电梯 (POJ2392)

$1S$ $64M$

【问题描述】

N $(1 \leq N \leq 400)$ 种石头搭高，每种石头高度 H_i $(1 \leq H_i \leq 100)$ 、限制高度 a_i $(1 \leq a_i \leq 40000)$ 、数量 c_i ($1 \leq c_i \leq 10)$ ，求能搭出的最大高度。

【样例输入】

```
3
7 40 3
5 23 8
2 52 6
```

【样例输出】

```
48
```

【追加说明】

5*2+7*3+2*6 合法

5*4+7*3 不合法

【样例解释】

5*3+7*3+2*6

【题目分析】

首先要观察出限制高度低的石头要先放入，这样才能保证之前的决定（石头的选用）对后续石头的选用没有影响，也就是无后效性，因此先按照限制高度 a_i 排

序。剩下的就是和之前例题差不多的背包DP思路。

状态：$dp[i][j]$ 表示前 i 种石头能否达到高度 j ，其中第一维可以省略

状态转移：$dp[i][j] = dp[i-1][j-a[i].h]$

初值：$dp[0] = 1$

终值：在所有满足 $dp[j] == 1$ 中找最大的 j 值

时间复杂度：$O(N \times a_i)$

【完整代码】

```
#include<cstdio>
#include<algorithm>
#include<cstring>
#include<iostream>
using namespace std;
struct node{
    int h,a,c;
}a[4100];
int n,dp[2][410000];
int hi,ai,ci,j;
bool cmp(node a,node b){
    return a.a<b.a;
}
int main(){
    scanf("%d",&n);
    int hsum=0;
    int count=0;
    for (int i=1;i<=n;i++){
        scanf("%d%d%d",&hi,&ai,&ci);//高度，限高，块数
        for (j=1;j<=ci;j++){
            count++;
            a[count].h=hi;
            a[count].a=ai;
            a[count].c=ci;
        }
        hsum+=hi*ci;
    }
    int t=0;
    dp[t][0]=1;
    sort(a+1,a+count+1,cmp);
    for (int i=1;i<=count;i++){
        t=1-t;
        for (int v=a[i].a;v>=0;v--){
            dp[t][v]=dp[1-t][v];
            if (!dp[t][v]&&v>=a[i].h&&dp[1-t][v-a[i].h])

                    dp[t][v]=1;
```

```
        }
    }

    for (int i=hsum;i>=0;i--)
        if (dp[t][i]){
            printf("%d \n",i);
            break;
        }
}
```

例题18-21：最小的双陆棋 (POJ3519)

【问题描述】

类似飞行棋的单人抛骰子游戏，求在T步之内走到终点的概率。格子标记为0到n，一开始棋子在方格0，终点是方格n，抛骰子次数和方格数一样多，骰子以相等的

概率生成1到6的整数，如果棋子超过了终点则会回退超出的格子数，例如棋子在n-3的格子，骰子抛出5，则棋子最终会在n-2，除了起点和终点的格子可能会有两

种标记：“L”，“B"。”L“表示暂停一轮，”B“表示棋子回到起点。

【输入】

$n\,T\,L\,B$

接着 L 行，每行一个数表示标记L的格子编号

接着 B 行，每行一个数表示标记B的格子编号

【输出】

T 步内成功走到终点的概率（精度误差控制在0.00001）

【题目分析】

回合制就很自然地把每一次抛骰子类比成取用背包物品，接着考虑状态设计，原则就是要涵盖所有的可能出现的情况、状态转移时所需信息，在这道题目中：一、

目前是第几轮；二、棋子的位置；三、到目前情况的概率值；四、能否进行状态转移得到解。

实际解题中，我们会遇到很多细节有变化的新题，可以用上述原则去一一尝试状态设计，如果不成功重来。

本题其中一种可行的设计：

状态：$f[i][j]$表示第i步走到j格的概率。

状态转移：

$f[i+1][j+k]+=f[i][j]/6\ (1<=k<=6)$

$if(b[j]==1)\ f[i+1][0]+=f[i][j]$

$if(a[j]==1)\ f[i+2][j+k]+=f[i][j]/6\ (1<=k<=6)$

遍历状态的顺序也有讲究，原因是 B 标记操作会改变人所在的格子编号，防止在一轮中被反复操作，要采用降序遍历的方法。另外某次抛骰子超出终点的处理方

法，只要举几个数字例子就容易想到的，加入人所在编号为 j ，抛出了 k ，最终的位置应该是 $2*N-j-k$ 。

初值：$dp[0][0]=1$

终值：$\sum_{i=0}^{T} dp[i][n]$ 这里要考虑清楚，任何一轮到达终点n都算成功。

时间复杂度：$O(T\times n)$

注意：由于精度问题，此题答案要用 $.f$ 输出才能对，是数据有问题。

【完整代码】

```
#include <stdio.h>
#include <string.h>
#include <iostream>
using namespace std;
double dp[110][110];
int a[110],b[110],tmp,N,T,L,B;
int main(){
    while (scanf("%d%d%d%d",&N,&T,&L,&B)!=EOF){
        if (N==0&&T==0&&L==0&&B==0) break;
        memset(dp,0,sizeof(dp));
        memset(a,0,sizeof(a));
        memset(b,0,sizeof(b));
        for (int i=1;i<=L;i++) {
            scanf("%d",&tmp);
            a[tmp]=1;
        }
        for (int i=1;i<=B;i++){
            scanf("%d",&tmp);
            b[tmp]=1;
        }
        dp[0][0]=1;
        for (int t=0;t<T;t++)
            for (int i=N-1;i>=0;i--){
                if (dp[t][i]==0) continue;
                if (b[i]==1)
                    dp[t][0]+=dp[t][i];
                else{
                    if (a[i]==1)
                        for (int k=1;k<=6;k++){
                            if (i+k<=N)
                                dp[t+2][i+k]+=dp[t][i]/6;
                            else
                                dp[t+2][2*N-i-k]+=dp[t][i]/6;
                        }
                    else
                        for (int k=1;k<=6;k++){
                            if (i+k<=N)
                                dp[t+1][i+k]+=dp[t][i]/6;
                            else
                                dp[t+1][2*N-i-k]+=dp[t][i]/6;
                        }

                }
```

```
            }
        double ans=0;
        for (int i=1;i<=T;i++)
            ans+=dp[i][N];
        printf("%.6lf\n",ans);
    }
}
```

例题18-22：项目 (POJ3459)

$2S$ $64M$

【问题描述】

某周某公司要完成 m 个项目。为此，公司最多可以雇用 n 个人，为期一周。每个外部员工都将花费公司一定的工资欧元，除非他/她所参与的项目没有完成，在这

种情况下，无需付款。

对于每个项目，在一周内有一个完成的概率，这个概率是关于参与该项目的员工人数的函数。这些概率以百分比 p_{ij} 给出，其中 $i\,(1 \le i \le m)$ 是项目的数量，j

是从事该项目的人数。当然，当没有人参与项目 i 时，概率 p_{i0} 为零。

如果项目 i 确实在一周内完成，公司获得一定奖励，单位为欧元，如果未能完成，该公司将支付罚款，单位也是欧元。

公司当然希望利润最大化，所以要寻找最佳员工人数。注意：如果一个人受雇，他/她只从事一个项目。

假设 $p(0 < p < 1)$ 是工作按时完成的概率，$E1$ 是奖励，$E2$ 为惩罚（负数），那么利润就是 $p \times E1 + (1-p) \times E2$ 。

【输入】

有多组数据，每组数据格式如下：

- 第一行一个整数 $m\,(1 \le m \le 100)$ 表示项目数。
- 第二行一个整数 n ，其中 $(0 \le n \le 100)$，可用员工的最大数量。
- 第三行一个整数表示员工工资 $(0 \le \text{工资} \le 1000)$ ，工资以单位是欧元。
- m 行，每行对应一个项目 i ，包含 n 个整数 $p_{i1}, p_{i2}, \ldots, p_{in}$（它们是百分比，单位是%，$0 \le p_i1, p_i2, \ldots, p_in \le 100$），接着两个整数表示对应于项目 i 的奖励和惩罚 $(0 \le \text{奖励，惩罚} \le 100000)$（单位是欧元），整数间都有空格分隔。

【输出】

对应每个输入文件输出两行：

- 第一行表示以欧分为单位的最大利润。
- 第二行表示为实现此最大利润而必须雇用的外部员工总数。如果最大的预期利润可以通过不同数量的员工实现，那么这些不同的数字必须按递增顺序给出。数字间用一个空格分隔。

【题目分析】

大概的题意就是 m 个项目雇佣 n 个工人完成，给定每个项目雇佣人数费用，以及每个项目雇佣人数完成概率，收益=成功概率 $\times$ 成功收益 $-$ 不成功概率 $\times$ 不成

功亏损，雇佣人另外要付每人输入给定的固定工资，不雇佣就不用付，一个人不能被重复使用，问最大收益。

有了前面例题的经验，继续用背包DP思路解题：

设计状态：$f[i][j]$ 表示完成 i 个项目用 j 个人的收益。

尝试状态转移：$f[i][j]=f[i-1][j-k]+cost(i,k)\ (0<=k<=j)$ k 表示第 i 个项目用 k 个人去完成。

由于输出是欧分，输入是欧元，因此概率的百分母刚好不用算，所有运算用给定的整数即可。

这道题目的初值需要考虑清楚：状态数组 f 要被赋最小值，$f[i][0]$ =第 i 个项目失败损失+ $f[i-1][0]$ 。

终值：$max\{f[m][i]\}(0\le i\le n)$

时间复杂度：$O(m\times n\times n)$

易错点：初值注意 $f[i][0]$ = $-100\times$ 项目失败亏损 $+f[i-1][0]$ ，$f[i][0]$ 相当于表示前 i 个项目共亏损了多少，另外统计最优值允许雇佣 0 个人的情况。

思考：初值从 0 个项目、1 个项目开始都可以，对应的初值应该如何设置。

【完整代码】

```
#include<cstdio>
#include<algorithm>
#include<iostream>
using namespace std;
const int inf=0x3fffffff;
int f[200][200],cost[200][200],a[200][200];
int main(){
    int T,m,n,v;
    scanf("%d",&T);
    for (int t=1;t<=T;t++){
        scanf("%d%d%d",&m,&n,&v);
        for (int i=0;i<=m;i++)
            for (int j=0;j<=n;j++)
                f[i][j]=-inf;
        for (int i=1;i<=m;i++){
            for (int j=1;j<=n+2;j++){
                scanf("%d",&a[i][j]);
            }
        }
        for (int i=1;i<=m;i++)
            for (int j=0;j<=n;j++)
                cost[i][j]=a[i][j]*(a[i][n+1]-v*j)-(100-a[i][j])*a[i][n+2];
        f[0][0]=0;
        for (int i=1;i<=m;i++)
            f[i][0]=-100*a[i][n+2]+f[i-1][0];
        for (int i=1;i<=m;i++)
            for (int j=1;j<=n;j++)
                for (int k=0;k<=j;k++)
                    f[i][j]=max(f[i][j],f[i-1][j-k]+cost[i][k]);
        int ans=-inf;
        for (int i=0;i<=n;i++)
            ans=max(ans,f[m][i]);
        printf("%d\n",ans);
```

```
        bool hav=0;
        for (int i=0;i<=n;i++)
            if (ans==f[m][i]){
                if (!hav) {
                    hav=1;
                    printf("%d",i);
                }
                else {
                    printf(" %d",i);
                }
            }
        printf("\n");
    }
}
```

例题18-23：蚂蚁数 (POJ3046)

【问题描述】

有 t 组 n 个蚂蚁，每组蚂蚁个数为 $a[i]$ ，同一组蚂蚁没有区别。求s~e只蚂蚁总共能形成的集合种数？

【输入】

$t\ n\ s\ e$

n 行，每行一个整数 X ，表示每个蚂蚁的组编号

【输出】

符合题意的集合种数

【样例输入】

```
3 5 2 3
1
2
2
1
3
```

【样例输出】

```
10
```

【样例解释】

3 种，1 只蚂蚁：{1} {2} {3} 5 种，2 只蚂蚁：{1,1} {1,2} {1,3} {2,2} { 2,3} 5 种，3 只蚂蚁： {1,1,2} {1,1,3} {1,2,2} {1,2,3} {2,2,3} 3 种 ，4 只蚂蚁: {1,2,2,3} {1,1,2,2} {1,1,2,3} 1 种 ，5 只蚂蚁：{1,1,2,2,3}

【题目分析】

本质题意就是n种数，形成m序列长度集合的方案数和。

状态：$dp[i][j]$ 表示前 i 种蚂蚁拿出 j 个的能形成的种类数。

状态转移：$dp[i][j]=\sum_{k=min(j,a[i])}^{0} dp[i-1][j-k]$ 。

但时间复杂度太高了，大概是$O(t*a*sum(a[i]))$，关于 k 存在一个优化，利用递推方程错开相减，最终可以加速达到 n^2 。

优化过程：

当 $j<=a[i]$,将其右边展开得到 $dp[i][j]=dp[i-1][j]+dp[i-1][j-1]+\cdots+dp[i-1][0]$。

将第一项 $dp[i-1][j]$ 单独抽离出来，就可以发现剩下的相当与 $dp[i][j-1]$ (根据一开始的转移方程)。

所以得到$dp[i][j]=dp[i-1][j]+dp[i][j-1]$

接下来考虑 $j>a[i]$,同样将其展开，右边变成下式：

$dp[i-1][j]+dp[i-1][j-1]+...+dp[i-1][j-a[i]]$

为了像 $j<=a[i]$ 时将后面凑出一个整项，在这个式子后再添加两项。变成如下：

$dp[i-1][j]+dp[i-1][j-1]+...+dp[i-1][j-a[i]]+dp[i-1][j-1-a[i]]-dp[i-1][j-1-a[i]]$

将第一项抽里出来，同样发现剩下的除了最后一项外，中间的所有项可以凑成 $d[i][j-1]$

最终的状态转移如下：

$dp[i][j]=dp[i-1][j]+dp[i][j-1]-dp[i-1][j-a[i]-1]$

时间复杂度降低到 $O(T\times n)$

初值：$dp[0][0]=1$ ，$dp[1][0]=1$

终值：$\sum_{i=s}^{e} dp[t][i]$

另外空间上可以用滚动数组优化。

【完整代码】

```
#include<cstdio>
using namespace std;
const int M=1e6;
int T,A,S,B,x,s,dp[3][1100000],c[10005];
int main(){
    scanf("%d%d%d%d",&T,&A,&S,&B);
    for (int i=1;i<=A;i++){
        scanf("%d",&x);
        c[x]++;
    }
    int t=0;
    dp[0][0]=1;dp[1][0]=1;
    for (int i=1;i<=T;i++){
        t=1-t;
        for (int j=1;j<=A;j++){ //j不能从0开始，想想为什么
            if(c[i]<j)dp[t][j]=(dp[1-t][j]+dp[t][j-1]-dp[1-t][j-c[i]-1]+M)%M;//加减法同余要注意负数，可以先加个M
            else dp[t][j]=(dp[1-t][j]+dp[t][j-1])%M;

        }
```

```
    }
    int ans=0;
    for (int i=S;i<=B;i++)
        ans=(ans+dp[t][i])%M;
    printf("%d\n",ans);
}
```

解决了这么多例题，接着是**实战演练**，请读者完成以下练习题：

例题18-24：不同质数之和 (POJ3132)

【问题描述】

$n\ (n \le 1120)$ 表示成 $k\ (k \le 14)$ 个素数和的方案数。

【解题思路】

$f[i][j]$ 表示已选择 i 个数，和为 j 的方案数

思考：for(int j=1;j<=m;j++) 改为 for (int j=m;j>=1;j--) 行不行，为什么？可以联系挤牛奶的例题。

【完整代码】

```
#include<cstdio>
#include<iostream>
#include<cstring>
using namespace std;
int f[20][2000],vis[2000],a[2000];
int main(){
    int c=0;
    for (int i=2;i<=1120;i++)
    {
        if (!vis[i]){
            a[++c]=i;
            for (int j=i;j<=1120;j+=i){
                if (!vis[j]) vis[j]=1;
            }
        }
    }
    int n,m;
    while (~scanf("%d%d",&n,&m)){
        if (n==0 && m==0) break;
        memset(f,0,sizeof(f));
        f[0][0]=1;
        for (int i=1;i<=c;i++)
            for (int j=m;j>=1;j--)
                for (int k=1;k<=n;k++)
                {
                    if (k>=a[i]&&f[j-1][k-a[i]])
                        f[j][k]+=f[j-1][k-a[i]];
                }
```

```
        printf("%d\n",f[m][n]);
    }
}
```

例题18-25：洗衣服 (POJ3211)

【题目描述】

n 种颜色的衣服，只能一种种洗完，甲乙同时洗某一种，求最少洗完的时间，输入给定颜色数量，和洗每一种颜色需要的时间（甲乙相同）。

【解题思路】

对每种衣服做背包DP，甲乙尽量均匀负责衣服的清洗，使总花费时间最接近，设计状态：$f[i]$ 表示前 n 种衣服任取能否组成 i 的时间，最终找到 $f[i] == 1$ 并且 i

尽可能接近总时间的一半。

【完整代码】

```
#include<cstdio>
#include<cstring>
#include<iostream>
#include<cmath>
#include<algorithm>
using namespace std;
char s[20][20];
int ans,num,sum,f[200000],b[200];
struct node{
    int t;
    char s[20];
}a[200];
int main(){
    int m,n;
    while (~scanf("%d%d",&m,&n)){
        if (m==0&&n==0)break;
        for (int i=1;i<=m;i++)
            scanf("%s",s[i]);
        for (int i=1;i<=n;i++)
            scanf("%d%s",&a[i].t,a[i].s);
        ans=0;
        for (int i=1;i<=m;i++){
            num=0;sum=0;
            for (int j=1;j<=n;j++)
            {
                if (strcmp(s[i],a[j].s)==0){
                    b[++num]=a[j].t;
                    sum+=a[j].t;
                }
            }
            memset(f,0,sizeof(f));

            f[0]=1;
```

```
            for (int j=1;j<=num;j++)
                for (int k=sum;k>=b[j];k--){
                    if (f[k-b[j]]) f[k]=1;
                }
            for (int j=sum/2;j>=0;j--)
                if (f[j]) {
                    ans+=abs(sum-j);
                    break;
                }

        }
        printf("%d\n",ans);
    }
}
```

例题 18-26：整除 (POJ1745)

【题目描述】

n 个数，插入 $n-1$ 个加号或者减号，看能否被 k 整除。

【解题思路】

$f[i][j]--->f[i+1][j-a[i]], f[i+1][j+a[i]]$，另外可以把负数转成同余正数来做：
$-m+k=(m\%k+k)\%k$

【完整代码】

```
#include<cstdio>
#include<algorithm>
using namespace std;
int f[11000][110],a[11000];
int main(){
    int n,k;
    scanf("%d%d",&n,&k);
    for (int i=1;i<=n;i++) scanf("%d",&a[i]);
    f[1][(a[1]%k+k)%k]=1;
    f[1][(-a[1]%k+k)%k]=1;
    for (int i=1;i<=n-1;i++)
        for (int j=0;j<=k-1;j++)
            if (f[i][j]) {
                f[i+1][((j-a[i+1])%k+k)%k]=1;
                f[i+1][((j+a[i+1])%k+k)%k]=1;
            }
    if (f[n][0]) printf("Divisible\n");else printf("Not divisible\n");
}
```

例题 18-27：取数 (POJ2184)

【题目描述】

n 对数任取，求取到的两对数和最大，并且每一对数非负。 **【解题思路】**

$f[i][j][k]$ 前 i 个数对能否取到 j 和 k ，但发现 $i \times j \times k$ 会超时，应该把 k 当作 $f[i][j]$ 状态的值

如上设计的易错点：考虑下方代码中的 $addv$ 是 1000 ，改成 1000×100 即可，因为表示和的 j 是 100 个数值和， 1000 不够

【完整代码】

```
#include<cstdio>
#include<iostream>
#include<cstring>
using namespace std;
const int addv=1000*100;
const int inf=0x3fffffff;
int f[2][300000],a[200],b[200];
int main(){
    int n;
    scanf("%d",&n);
    int sum1f=0,sum1z=0;
    for (int i=1;i<=n;i++){
        scanf("%d%d",&a[i],&b[i]);
        if (a[i]<0) sum1f+=a[i];else sum1z+=a[i];
    }
    for (int j=0;j<300000;j++){
        f[0][j]=-inf;f[1][j]=-inf;
    }
    f[0][addv]=0;
    int c=0;
    for (int i=1;i<=n;i++){
        c=1-c;
        for (int j=sum1f;j<=sum1z;j++){
            f[c][j+addv]=max(f[c][j+addv],f[1-c][j+addv]);
            f[c][j+addv]=max(f[c][j+addv],f[1-c][j-a[i]+addv]+b[i]);
        }
    }
    int ans=-1;
    for (int j=0;j<=sum1z;j++)
        if (f[c][j+addv]>=0)
            ans=max(ans,f[c][j+addv]+j);
    printf("%d\n",ans);
}
```

18.2 小结

|背包型DP的定义：给定一个有体积的背包和一些物品以及特定限制，求解最优物品选择方案。

|时间复杂度：一般情况下，可以通过状态数×状态转移操作数来求取。

|背包型DP功能：在回合制决策、分阶段摆放物品等问题模型中，求解时间要远远快于穷举类的算法。

|背包型DP设计原则：状态是否能涵盖所有情况、是否涵盖状态转移所需信息。

|背包型DP初值、终值必须考虑仔细，不然状态设计再正确也将是错误答案。

背包型DP有很多的适用场合，是一个特别高效的算法，也可以结合数据结构、图论算法来解决更多的复杂问题。